Les services

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Services proposés par Symfony **3**

II. Exercice : Exercice applicatif : Utiliser le service LoggerInterface **7**

III. Création des services de l'application **7**

IV. Exercice : Exercice applicatif : Un service de réduction de prix **10**

V. Essentiel **10**

VI. Auto-évaluation **10**

A. Exercice : 10

B. Test 10

Solutions des exercices **12**

Tous droits réservés à STUDI - Reproduction interdite

I. Services proposés par Symfony

Durée : 1 h

Environnement de travail : Visual Studio Code (ou tout autre éditeur de texte), PHP, Composer, Maker bundle, Doctrine

Pré-requis : PHP, Doctrine, Maker bundle

Contexte

Beaucoup de **frameworks** se basent sur la notion de **service.** Dans le cas de **Symfony,** nous pouvons voir les **services** comme une couche supplémentaire au **pattern Modèle Vue Contrôleur (MVC).**

Les **services** peuvent fournir des outils complémentaires et réutilisables mais aussi apporter une logique métier à vos applications web **Symfony.**

C'est donc dans ces **services** que doivent se trouver les **classes** constituantes de la **logique métier** de l'application.

Attention

Ce qui est appelé « service » dans Symfony ne correspond pas à ce que nous appelons service (dans le sens de fonctionnalité d'un pattern client-server) en ingénierie informatique.

Objectifs

- Découvrir les **services** de **Symfony**
- Utiliser un **service** proposé par **Symfony**
- Utiliser **l'injection de dépendance**
- Découvrir le **conteneur de services**
- Comprendre la notion d'**autowiring**

Contexte

En réalité, à l'initialisation même d'un nouveau projet **Symfony,** celui-ci contient déjà de nombreux **services.** Vous pouvez voir cela comme une **boite à outils** prêts à être utilisés fournie par **Symfony.**

Aujourd'hui, utiliser un **service** au sein d'une application se fait, d'ailleurs, très simplement grâce à une notion que nous détaillerons davantage dans ce cours : **l'injection de dépendance.**

Dans cette partie, nous allons donc nous intéresser à quelques **services** proposés par **Symfony** avant de découvrir, par la pratique, comment **injecter** un tel **service** dans un **contrôleur.**

Un **service** correspond donc à une **fonctionnalité** préalablement codée et **réutilisable** dans l'ensemble de notre application **Symfony.** Plus concrètement, un **service** est représenté par une **classe PHP.**

Pour comprendre comment fonctionne un **service,** il faut comprendre le **conteneur de services.** En réalité, un **service** ne pourrait être utilisé, en dehors de la **classe** où il a été implémenté, sans ce **conteneur.**

Il permet de créer et configurer les **services** qu'il contient afin qu'ils soient disponibles et prêts à l'emploi dans un projet **Symfony** et ce, dès sa création.

Tous droits réservés à STUDI - Reproduction interdite

Pour découvrir tous les **services** disponibles, **Symfony** propose d'utiliser la **commande** `$ php bin/console debug:autowiring`.

```
Chris_Chevalier@Macbook project % php bin/console debug:autowiring

Autowirable Types
=================

 The following classes & interfaces can be used as type-hints when autowiring:

 App\Kernel (kernel)

 CacheItemPoolInterface generates CacheItemInterface objects.
 Psr\Cache\CacheItemPoolInterface (cache.app)

 Describes the interface of a container that exposes methods to read its entries.
 Psr\Container\ContainerInterface (service_container) - deprecated
 Psr\Container\ContainerInterface $parameterBag (parameter_bag)

 Defines a dispatcher for events.
 Psr\EventDispatcher\EventDispatcherInterface (event_dispatcher)

 Describes a logger instance.
 Psr\Log\LoggerInterface (logger)

 SessionHandlerInterface (session.handler.native_file)

 Interface for adapters managing instances of Symfony's CacheItem.
 Symfony\Component\Cache\Adapter\AdapterInterface (cache.app)

 ContainerInterface is the interface implemented by service container classes.
 Symfony\Component\DependencyInjection\ContainerInterface (service_container) - deprecated

 ContainerBagInterface is the interface implemented by objects that manage service container parameters.
 Symfony\Component\DependencyInjection\ParameterBag\ContainerBagInterface (parameter_bag)
```

Voici le résultat ainsi obtenu en **console** :

Notez que cette capture d'écran ne contient pas l'ensemble des **services** proposés par **Symfony.**

Pour utiliser un **service,** on ne va pas appeler le **conteneur** mais bien la **classe du service** qu'il contient en **l'injectant** directement en **paramètre** de la **méthode** dans laquelle nous souhaitons l'utiliser.

Nous parlons d'**injection de dépendance.** En effet, passer un **service** en **paramètre** d'une **méthode** le rend utilisable librement dans ladite **méthode** sans traitement supplémentaire. Cette pratique est rendue possible grâce au mécanisme de ***l'autowiring.***

Méthode

Voyons, sans plus tarder, comment **injecter une dépendance** au sein d'un **contrôleur.**

Imaginons un **contrôleur** permettant d'afficher tous les produits stockés dans une **base de données.**

Exemple **Code**

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\ProductRepository;

class ProductController extends AbstractController {
    /**
     * @Route("/", name="product")
     */
```


Tous droits réservés à STUDI - Reproduction interdite

```
13      public function index(ProductRepository $repository) {
14          $products = $repository->findAll();
15          return $this->render("product.html.twig", [
16              "products" => $products
17          ]);
18      }
19  }
20
21  ?>
```

Pour récupérer tous les produits stockés en **base de données,** il est nécessaire d'appeler la **méthode** `findAll()`, issue de son **repository.** En réalité, `ProductRepository` est un **service** que nous pouvons **injecter** dans notre **contrôleur.**

Pour ce faire, il suffit de passer une **instance** de la **classe du service** souhaité en **paramètre** de la **méthode** `index()` de notre **contrôleur.** De cette manière, le **service** est accessible dans l'ensemble de la **méthode.** Nous pouvons ainsi appeler `findAll()` très facilement.

Cette pratique peut paraître magique mais, en réalité, elle est rendue possible grâce au **conteneur de services.** Pour rappel, il s'agit d'une **boîte à outils** présente dans tous vos projets **Symfony.** En l'occurence, `ProductRepository` représente un des **services** disponibles dans le **conteneur.**

Ce **conteneur** contient bien d'autres services comme un **système de gestion de mails,** un **système de gestion de routes,** un **outil pour se connecter à une base de données** mais également **le moteur de template Twig,** entre autres.

Ainsi, si vous avez besoin de l'un de ces outils, il suffit d'**injecter la dépendance** comme vous venez de le faire avec `ProductRepository`. Le **conteneur** comprendra automatiquement ce dont vous avez besoin grâce au **paramètre** passé à la **méthode.** Cette pratique s'appelle **autowiring.**

Grâce à cette notion, le **conteneur de services** va se charger de construire une **instance** de la **classe** demandée s'il la connaît. En l'occurrence, c'est lui qui a créé l'**instance** `$repository` issue de la **classe** `ProductRepository`. Il a donc compris cette **classe** pour en construire une **instance** à partir de son **constructeur,** bien sûr.

En bref, le **conteneur de services** représente une **boîte à outils** contenant des **classes PHP** qu'il peut comprendre pour les **instancier** sur demande.

Pour **injecter un service** de cette manière, il suffit de savoir s'il existe grâce à la **commande** `$ php bin/console debug:autowiring`. Tous les **services** retournés par cette simple commande sont accessibles exactement de la même manière dans vos **classes.**

Vous pouvez néanmoins remarquer que le `ProductRepository` n'apparaît pas en sortie de cette **commande.** En réalité, elle est censée afficher uniquement les **services** rendus disponibles par **Symfony.** Or, c'est nous qui avons créé automatiquement ce **service** en utilisant la **commande** `$ php bin/console make:entity`.

Ainsi, pour obtenir la liste de **tous les services,** mêmes les nôtres, il suffit d'ajouter le **drapeau** `--all` à la **commande d'autowiring.**

Tous droits réservés à STUDI - Reproduction interdite

```
Chris_Chevalier@Macbook project % php bin/console debug:autowiring --all

Autowirable Types
=================

 The following classes & interfaces can be used as type-hints when autowiring:

 App\Controller\ProductController

 App\DataFixtures\AppFixtures

 App\DataFixtures\ProductFixtures

 App\Kernel (kernel)

 App\Repository\ProductRepository

 Doctrine\Bundle\DoctrineBundle\Dbal\ManagerRegistryAwareConnectionProvider

 Interface for annotation readers.
Doctrine\Common\Annotations\Reader (annotations.cached_reader)
```

Nous découvrons ici que toutes les **classes** que nous avons créées jusqu'ici en font partie et sont donc accessibles depuis n'importe où.

Cet ajout a été rendu possible grâce à la **configuration** du **conteneur de services** disponible dans le fichier `./config/services.yaml`.

```yaml
parameters:

services:
    # default configuration for services in *this* file
    _defaults:
        autowire: true      # Automatically injects dependencies in your services.
        autoconfigure: true # Automatically registers your services as commands, e

    # makes classes in src/ available to be used as services
    # this creates a service per class whose id is the fully-qualified class name
    App\:
        resource: '../src/'
        exclude:
            - '../src/DependencyInjection/'
            - '../src/Entity/'
            - '../src/Kernel.php'
            - '../src/Tests/'
```

Dans la section `services` du fichier, nous découvrons que, par défaut, **Symfony** a configuré la **propriété** `autowire` à `true`. Cette **configuration** permet de rendre automatique **l'injection de dépendances.** Dans la même idée, la **propriété** `autoconfigure` à `true` permet de charger automatiquement les **services.**

Plus bas dans le code **YAML,** nous découvrons que toutes les **classes** se trouvant dans le dossier `./src` peuvent être utilisées en tant que service, à l'exception de celles se trouvant dans les dossiers **DependencyInjection, Entity, Migrations, Tests** et du fichier **Kernel.php.**

Tous droits réservés à STUDI - Reproduction interdite

Cela signifie que le **conteneur de services** va charger et s'injecter toutes les **classes** qui se trouvent dans le dossier `./src` au chargement de l'application **Symfony.**

Syntaxe **À retenir**

Cette partie du cours contient beaucoup de **théorie,** je vous invite, néanmoins, à retenir certains éléments de **syntaxe.** La **commande** `$ php bin/console debug:autowiring` va vous être utile tout au long de votre carrière de développeur **Symfony** étant donné qu'elle permet d'afficher en **console** tous les **services** proposés par le framework.

Associée au **drapeau** `—all`, elle renvoie également les **classes,** situées dans le dossier `./src`, automatiquement ajoutées au **conteneur de services.**

Ce mécanisme est rendu possible par la **configuration** du **conteneur** disponible et modifiable depuis le fichier `./config/services.yaml` de votre projet **Symfony.**

II. Exercice : Exercice applicatif : Utiliser le service LoggerInterface

Question

Nous avons vu comment utiliser un **service** que nous avons nous même créé indirectement. Pour cet exercice pratique, nous vous demandons d'utiliser un **service** directement proposé par **Symfony** en suivant la démarche mise en place pour utiliser le `ProductRepository` au sein de notre **méthode** `index()`.

Si ça n'est pas déjà fait, créez une **entité** `Product` à l'aide du bundle **Maker** puis implémentez le code de `ProductController`, disponible plus haut. Configurez une **base de données** contenant quelques produits ; vous pouvez utiliser une **fixture** si vous l'estimez nécessaire.

Ceci étant fait, imaginons que le nom du premier produit n'apparaît pas correctement dans votre **navigateur web.** Aussi, vous souhaitez **déboguer** votre application pour comprendre ce que contient la **variable** censée stocker le nom du produit.

Pour ce faire, **injectez le service** `LoggerInterface` à votre **méthode** `index()`. Notez que la **méthode** `info()`, issue de cette **classe**, permet justement d'afficher les informations qui lui sont passées en **argument** en **console.**

Indice :

Vous trouverez **l'espace de nommage** du **service,** utile à l'importation, à l'aide de la **commande** `$ php bin/console debug:autowiring`.

III. Création des services de l'application

Objectifs

- Créer son propre **service**
- Utiliser son propre **service**
- Comprendre le **conteneur de services**

Contexte

Dans cette partie du cours, nous allons nous intéresser à la création de **service.** Pour rappel, c'est dans ce genre de **service** que nous allons vouloir implémenter la **logique métier.** Bien entendu, nos **services** représentent des **classes.** Ainsi, l'orienté objet est à la base de l'utilisation des **services.**

Nous allons ainsi pouvoir faire **hériter** nos **services** de **classes mères** ou encore les faire **implémenter** des **interfaces.**

Tous droits réservés à STUDI - Reproduction interdite

À terme, l'idée sera d'utiliser et réutiliser nos **services** au sein de nos **contrôleurs** afin d'éviter la duplication de code et ainsi construire des applications robustes, évolutives et maintenables dans le temps.

Attention

Le principe d'un service est d'être **indépendant de l'orientation de son client**.

Nous pouvons implémenter un service en fonctionnel et l'utiliser en objet ou l'inverse.

Nous pourrions aussi bien exposer une fonction qui retourne un objet lambda, sans faire d'OO, ce serait toujours un service.

Ici, « service » est à comprendre comme « **service Symfony** ». Il ne faut pas confondre avec l'idée d'implémenter un service en utilisant Symfony.

Reprenons notre `ProductController`. Imaginons désormais que nous souhaitons abstraire l'affichage de nos produits afin de ne pas avoir à boucler dessus depuis notre template **Twig.** Pour ce faire, nous pouvons imaginer un **service** permettant un tel affichage.

Méthode

Voyons comment on pourrait écrire ce **service.**

Exemple **Code : PHP**

```php
<?php

namespace App\Service;

class ProductService
{
    public function displayProducts($products): string {
        $result = "";
        foreach ($products as $product) {
            $result .=  " - {$product->getName()} : {$product->getPrice()} €";
        }
        return $result;
    }
}

?>
```

Pour résumer, le `ProductService` représente une **classe** très simple, située dans le dossier `Service`, créé au préalable et contenant une unique **méthode** `displayProducts()`. Cette dernière prend un tableau de produit en **paramètre** et renvoie une **chaîne de caractères** contenant la concaténation des différents produits avec leur **nom** et leur **prix.**

Pas mal... Il nous faut maintenant l'utiliser dans notre **contrôleur.** Souvenez-vous, on a découvert, dans la partie précédente, qu'une **classe** appartenant au dossier `./src` est automatiquement ajoutée au **conteneur de services** en tant que... **service,** bien sûr.

Vérifions cela avec la fameuse **commande** `$ php bin/console debug:autowiring —all`, sans oublier le **drapeau** `--all` qui, je vous le rappelle, permet d'afficher tous les **services** et non plus uniquement ceux proposés par **Symfony.**

Tous droits réservés à STUDI - Reproduction interdite

```
Chris_Chevalier@Macbook project % php bin/console debug:autowiring --all

Autowirable Types
=================

 The following classes & interfaces can be used as type-hints when autowiring:

 App\Controller\ProductController

 App\DataFixtures\AppFixtures

 App\DataFixtures\ProductFixtures

 App\Kernel (kernel)

 App\Repository\ProductRepository

 App\Service\ProductService
```

Super ! Le service `ProductService` a déjà été ajouté au **conteneur** !

On peut alors l'utiliser comme n'importe quel autre **service**. Voyons comment l'intégrer correctement à mon `ProductController`.

Exemple **Code : PHP**

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\ProductRepository;
use App\Service\ProductService;

class ProductController extends AbstractController {
    /**
     * @Route("/", name="product")
     */
    public function index(ProductRepository $productRepository, ProductService $productService) {
        $products = $productRepository->findAll();
        return $this->render("product.html.twig", [
            "products" => $productService->displayProducts($products),
        ]);
    }
}

?>
```

Comme `ProductRepository`, `ProductService` est d'abord **importé** avant d'être **injecté** en **paramètre** de la **méthode** `index()` de `ProductController`. Nous sommes ensuite libre d'utiliser **l'instance** ainsi créée par le **conteneur de services** au sein même de la **méthode.** En l'occurrence, nous l'utilisons pour faire passer la **chaîne de caractères** qu'elle renvoie en tant que **variable** de notre template **Twig.**

Tous droits réservés à STUDI - Reproduction interdite

Syntaxe **À retenir**

Concernant cette partie du cours, retenez simplement qu'un **service** est représenté par une **classe** située dans le dossier `./src` de notre projet **Symfony.**

Enfin, souvenez-vous que l'on peut utiliser la **commande** `$ php bin/console debug:autowiring --all` pour vérifier si nos propres **services** apparaissent bien dans la sortie de la **console.**

IV. Exercice : Exercice applicatif : Un service de réduction de prix

Question

Créez un **service** `ReductionService` dont l'unique **méthode** `applyReduction()` applique une réduction à un produit. Cette **méthode** reçoit en **argument** un produit ainsi que le montant de la réduction et retourne un **entier** représentant le nouveau prix après application de la réduction.

Injectez ce **service** dans la **méthode** `index()` de `ProductController` pour tester une réduction du prix du premier produit enregistré en **base de données.** Mettez à jour votre template **Twig** pour confirmer le bon fonctionnement du **service** directement depuis le navigateur.

V. Essentiel

VI. Auto-évaluation

A. Exercice :

Question

Pour ce faire, créez un **contrôleur** `DeleteProductController`, pointant sur la **route** `/delete/{id}`, chargé de **supprimer** le produit correspondant à l'**id** passé en paramètre de l'**URL** avant de **rediriger** l'utilisateur vers la **route racine (/)** de votre projet.

À vous d'utiliser correctement votre **service** `ProductRepository` pour retrouver le produit à supprimer avec son **id** ainsi que le **service** `EntityManagerInterface`, proposé par **Symfony,** pour supprimer l'objet et synchroniser la suppression avec la **base de données.**

Profitez-en pour mettre à jour le template **Twig** afin que l'on puisse supprimer un produit en cliquant sur son nom ou sur une icône, par exemple.

B. Test

Exercice : Quiz

Question 1

En **Symfony,** quel concept représente une couche supplémentaire au **pattern Modèle Vue Contrôleur** ?

- ○ Les templates
- ○ Les services
- ○ Les entités
- ○ Les test

Question 2

Qu'apportent les services à vos applications **Symfony** ?

Tous droits réservés à STUDI - Reproduction interdite

- Des templates
- Plus de sécurité
- Une logique orientée composant
- Une logique métier

Question 3

À sa création, que contient déjà un projet **Symfony** ?

- Plusieurs services
- Plusieurs tests
- Plusieurs entités
- Plusieurs contrôleurs

Question 4

En **Symfony,** par quel élément de code est représenté un **service** ?

- Une constante
- Une variable
- Une méthode
- Une classe PHP

Question 5

Quelle **commande** permet d'obtenir tous les **services** proposés par **Symfony** ?

- `$ php bin/console debug:router`
- `$ php bin/console debug:autowiring`
- `$ php bin/console doctrine:database:create`
- `$ php bin/console debug:schema:update`

Question 6

Quelle notion permet d'utiliser un **service** au sein d'une **méthode** ?

- L'injection de dépendance
- Le Test Driven Development
- L'encapsulation
- L'abstraction

Question 7

Quel **drapeau** doit-on ajouter à la **commande d'autowiring** pour obtenir également les **services** que l'on a créés ?

- `--dev`
- `--filter`

Tous droits réservés à STUDI - Reproduction interdite

- `--all`
- `--auto`

Question 8

Dans quel dossier de votre projet **Symfony** doivent se trouver vos **services** pour être ajoutés au **conteneur de services** automatiquement ?

- `./config`
- `./Controller`
- `./Service`
- `./src`

Question 9

Dans quel fichier, pouvons-nous configurer le **conteneur de services** ?

- `./config/routes.yaml`
- `./config/services.yaml`
- `./src/Service/services.yaml`
- `./public/services.yaml`

Question 10

Quelle notion permet au **conteneur de services** de construire une **instance** de la **classe injectée** ?

- L'autowiring
- Le Test Driven Development
- L'encapsulation
- L'injection de dépendance

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. Solution n°1

Classe contrôleur

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\ProductRepository;
use Psr\Log\LoggerInterface;

class ProductController extends AbstractController {
    /**
     * @Route("/", name="product")
     */
    public function index(ProductRepository $productRepository, LoggerInterface $logger) {
        $products = $productRepository->findAll();
        $logger->info($products[0]->getName());
        return $this->render("product.html.twig", [
            "products" => $products
        ]);
    }
}

?>
```

Exercice p. Solution n°2

Classe service

```php
<?php

namespace App\Service;

class PriceReduction
{
    public function applyReduction($product, $priceReduction): int {
        return $product->getPrice() - $priceReduction;
    }
}

?>
```

Classe contrôleur

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\ProductRepository;
use App\Service\ReductionService;
use App\Service\ProductService;
```

Tous droits réservés à STUDI - Reproduction interdite

```php

class ProductController extends AbstractController {
    /**
     * @Route("/", name="product")
     */
    public function index(ProductRepository $productRepository, ProductService $productService, ReductionService $reductionService) {
        $products = $productRepository->findAll();
        return $this->render("product.html.twig", [
            "products" => $productService->displayProducts($products),
            "promoProduct" => $products[0],
            "promoPrice" => $reductionService->applyReduction($products[0], 1)
        ]);
    }
}

?>
```

Template Twig associé

```twig
{% extends "base.html.twig" %}

{% block body %}
    <h2>Nos produits</h2>
    {{ products }}

    <i>Le produit {{ promoProduct.name }} a un prix réduit de {{ promoPrice }}</i>
{% endblock %}
```

Exercice p. Solution n°3

Classe contrôleur

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Routing\Annotation\Route;
use App\Repository\ProductRepository;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Component\HttpFoundation\RedirectResponse;

class DeleteProductController extends AbstractController {
    /**
     * @Route("/delete/{id}")
     */
    public function index(ProductRepository $productRepository, int $id, EntityManagerInterface $em): RedirectResponse {
        $product = $productRepository->find($id);
        $em->remove($product);
        $em->flush();
        return $this->redirect("/");
    }
}

?>
```


Tous droits réservés à STUDI - Reproduction interdite

Template Twig

```
{% extends "base.html.twig" %}

{% block body %}
    <h2>Nos produits</h2>
    {% for product in products %}
        <p> => <a href="/delete/{{product.id}}">{{ product.name }}</a> : {{ product.price }} €</p>
    {% else %}
        <b>Aucun produit pour le moment</b>
    {% endfor %}
{% endblock %}
```

Exercice p. 10 Solution n°4

Question 1

En **Symfony,** quel concept représente une couche supplémentaire au **pattern Modèle Vue Contrôleur** ?

- ○ Les templates
- ◉ Les services
- ○ Les entités
- ○ Les test

Dans le cas de Symfony, nous pouvons voir les services comme une couche supplémentaire au pattern Modèle Vue Contrôleur.

Question 2

Qu'apportent les services à vos applications **Symfony** ?

- ○ Des templates
- ○ Plus de sécurité
- ○ Une logique orientée composant
- ◉ Une logique métier

Les services correspondent à des classes constituantes de la logique métier de l'application.

Question 3

À sa création, que contient déjà un projet **Symfony** ?

- ◉ Plusieurs services
- ○ Plusieurs tests
- ○ Plusieurs entités
- ○ Plusieurs contrôleurs

À l'initialisation d'un projet Symfony, celui-ci contient déjà de nombreux services. Vous pouvez voir ça comme une boîte à outils prêts à être utilisés, fournie par Symfony.

Question 4

En **Symfony,** par quel élément de code est représenté un **service** ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Une constante
- ○ Une variable
- ○ Une méthode
- ◉ Une classe PHP

Concrètement, un service est représenté par une classe PHP.

Question 5

Quelle **commande** permet d'obtenir tous les **services** proposés par **Symfony** ?

- ○ `$ php bin/console debug:router`
- ◉ `$ php bin/console debug:autowiring`
- ○ `$ php bin/console doctrine:database:create`
- ○ `$ php bin/console debug:schema:update`

Pour découvrir tous les services disponibles, Symfony propose d'utiliser la commande `$ php bin/console debug:autowiring`.

Question 6

Quelle notion permet d'utiliser un **service** au sein d'une **méthode** ?

- ◉ L'injection de dépendance
- ○ Le Test Driven Development
- ○ L'encapsulation
- ○ L'abstraction

Pour utiliser un service, nous appelons sa classe du service en l'injectant directement en paramètre de la méthode dans laquelle nous souhaitons y faire appel. Nous parlons d'injection de dépendance.

Question 7

Quel **drapeau** doit-on ajouter à la **commande d'autowiring** pour obtenir également les **services** que l'on a créés ?

- ○ `--dev`
- ○ `--filter`
- ◉ `--all`
- ○ `--auto`

Pour obtenir la liste de tous les services, mêmes les nôtres, il suffit d'ajouter le drapeau `--all` à la commande d'autowiring.

Question 8

Dans quel dossier de votre projet **Symfony** doivent se trouver vos **services** pour être ajoutés au **conteneur de services** automatiquement ?

- ○ `./config`
- ○ `./Controller`

Tous droits réservés à STUDI - Reproduction interdite

- ○ `./Service`
- ◉ `./src`

Toutes les classes se trouvant dans le dossier `./src` peuvent être utilisées en tant que service.

Question 9

Dans quel fichier, pouvons-nous configurer le **conteneur de services** ?

- ○ `./config/routes.yaml`
- ◉ `./config/services.yaml`
- ○ `./src/Service/services.yaml`
- ○ `./public/services.yaml`

La configuration du conteneur de services est disponible dans le fichier `./config/services.yaml`.

Question 10

Quelle notion permet au **conteneur de services** de construire une **instance** de la **classe injectée** ?

- ◉ L'autowiring
- ○ Le Test Driven Development
- ○ L'encapsulation
- ○ L'injection de dépendance

Grâce à l'autowiring, le conteneur de services construit une instance de la classe demandée s'il la connaît.

Tous droits réservés à STUDI - Reproduction interdite